Powered by GPT - Software Quality Verification

Formatting Note

Apply the following formatting in the final Word document:

- font: `Times New Roman`
- font size: `12 pt` for body, `14 pt` for main headings
- line spacing: `1.5`
- standard margins
- automatic table of contents using Word heading styles

Cover Page Content

Powered by GPT - Software Quality Verification
Final Report for Software Testing Course

- System Under Test: `Online Sales Management System`
- Repository: `https://github.com/Alucard30Dec/Online-Sales-Management-System`
- Appendix Link: `https://github.com/Alucard30Dec/Online-Sales-Management-System/tree/main/Report%20Test%20subject`
- Team:
  - Hoang Van Thien - `22D1ITE-SWE03` - `225051915`
  - Nguyen Thanh Dat - `22D1ITE-SWE03` - `225050896`
  - Le Quang Duy - `22D1ITE-SWE03` - `225051169`
- Submission Date: `April 06, 2026`

Record Of Changes

Version	Date	Author	Change Summary
`0.1`	`2026-04-05`	QA team	Completed project audit, submission structure, and test plan baseline
`0.5`	`2026-04-05`	QA team	Added UI test cases, API test cases, test data, and automation design
`0.8`	`2026-04-05`	QA team	Added automation implementation, execution evidence, defect workflow, and metrics
`1.0`	`2026-04-06`	Hoang Van Thien	Consolidated final report content, analysis, and appendix linkage

Table Of Contents

Insert an automatic Word table of contents here after applying heading styles.

I. Overview

1.1 Project Information

This report presents the final software-testing submission for the `Online Sales Management System`, an `ASP.NET Core MVC (.NET 8)` application that manages authentication, products, suppliers, customers, purchases, invoices, stock, reports, and a public product catalog. The

project also exposes a small API surface for service health and public catalog access. The test submission was built directly from source-code inspection, seeded demo data, local execution, and evidence collected from the running application.

The main objective of this report is to verify whether the project behaves correctly for its business-critical workflows and whether the resulting artifacts satisfy the course rubric in test planning, test design, execution evidence, reporting quality, and automation readiness.

1.2 System Under Test

- Project name: `Online Sales Management System`
- Technology stack: `ASP.NET Core MVC (.NET 8)`, `EF Core`, `ASP.NET Identity`, `TiDB/MySQL`
- Test surfaces:
  - `Admin UI`
  - `Public Product Catalog`
  - `GET /api/v1/health`
  - `GET /api/v1/catalog/*`

1.3 Project Team And Task Allocation

The project was completed by a three-member QA team. Work allocation was designed so each member owned more than ten UI test cases and covered separate functional areas to avoid unnecessary overlap. `Hoang Van Thien` handled a slightly larger portion of the workload and also led the integration of planning, automation direction, and final report consolidation.

Team allocation

- `Hoang Van Thien`
  - led project audit and final integration
  - owned `17` UI test cases
  - main modules: `Authentication`, `Permissions`, `Admin Users`, `Admin Groups`, `Invoices`, `Reports`, partial `Public Catalog`
- `Nguyen Thanh Dat`
  - owned `13` UI test cases
  - main modules: `Customers`, `Suppliers`, `Purchases`, partial `Stock`
- `Le Quang Duy`
  - owned `14` UI test cases
  - main modules: `Products`, `Product Import`, partial `Stock`, partial `Public Catalog`

This allocation satisfies the requirement that each member contributes at least ten test cases while keeping role ownership visible for defense and review.

[INSERT TABLE O-1: summary ownership counts derived from `test-cases/ui/OSMS-UI-Test-Cases.xlsx`]

II. Test Plan

2.1 Test Scope

The testing scope focused on the highest-risk business and control areas of the project:

- authentication and login validation
- role-based access control for `admin`, `sales`, and `warehouse`
- product and master-data management
- purchase creation, receiving, and cancellation
- invoice creation, payment flow, and cancellation
- stock visibility and movement history
- report filtering and summary totals
- public catalog browsing, filtering, sorting, and details
- health and catalog API behavior

The following items remained outside scope for this submission:

- performance and load testing
- deep security penetration testing
- mobile testing
- full responsive certification across multiple devices
- unrevealed write APIs outside the exposed public catalog and health endpoints

2.2 Test Strategy And Approach

The project used a mixed strategy of `manual testing`, `API testing`, and `targeted automation`. Manual testing was prioritized for broad business-rule coverage, permission behavior, validation feedback, and human verification of visible outcomes. API testing was used for stable request-response validation on the exposed endpoints. Automation was applied selectively to high-value flows to maximize bonus potential without overextending the scope into unstable or low-value areas.

Execution followed a `black-box` approach, while test design used `white-box-informed` analysis of the source code to identify hidden edge cases, validation rules, route behavior, and permission boundaries.

2.3 Test Environment

The verified execution environment was:

- OS: `Windows 11`
- Runtime: `.NET 8`
- Browser verified: `Google Chrome`
- Local base URL: `http://localhost:5068`
- Repository path: `E:\Project\Online-Sales-Management-System`
- Database environment tag observed in UI: `TiDB / test`

The test data environment relied on seeded demo data from `Data/DbSeeder.cs`, including valid accounts, products, suppliers, customers, purchases, invoices, expenses, and stock movement records.

2.4 Tools Used

- planning and traceability: Markdown + Excel-compatible CSV/XLSX artifacts
- UI testing: manual browser testing + `Selenium WebDriver` with `.NET 8` and `xUnit`
- API testing: `Postman` and `Newman`
- bug management preparation: `GitHub Issues` taxonomy and defect register
- evidence and metrics: screenshots, TRX runner output, Newman output, CSV/XLSX metrics packs

[INSERT TABLE P-1: concise test-environment table based on `docs/test-plan.md`]

III. Test Design And Execution

3.1 Test Scenario Design

The scenario design phase identified `42` documented scenarios across UI and API surfaces. These scenarios were grouped by module and risk type, including positive flows, negative flows, validation flows, permission checks, and business-rule edge cases. The design intentionally emphasized critical modules such as authentication, permissions, purchases, invoices, stock, reports, and product import.

At the current stage, all `42` documented scenarios are represented in the current test-case files, which gives a scenario design coverage of `100%`. This closes the earlier traceability gap and strengthens the design section against rubric checks for completeness and coverage.

[INSERT TABLE D-1: scenario summary from `docs/test-scenarios.md` or `metrics/OSMS-Scenario-Coverage.csv`]

3.2 UI Test Case Design

The UI test suite contains `44` test cases covering:

- authentication
- permissions
- admin users and admin groups
- customers
- suppliers
- products
- product import
- purchases
- invoices
- stock
- reports
- public catalog

Each UI test case includes:

- Test Case ID
- Scenario ID

- Title
- Module
- Preconditions
- Test Data
- Steps
- Expected Result
- Actual Result
- Status
- Owner

The test cases were written to maximize clarity, reproducibility, and rubric coverage. Negative cases were prioritized for authentication, permissions, validation, duplicate inputs, import restrictions, and inventory-related workflows.

[INSERT TABLE D-2: sample UI test case rows from `test-cases/ui/OSMS-UI-Test-Cases.xlsx`]

3.3 API Test Case Design

The API test suite contains `19` cases mapped to the real endpoints found in the source code:

- `GET /api/v1/health`
- `GET /api/v1/catalog/products`
- `GET /api/v1/catalog/products/{id}`
- `GET /api/v1/catalog/trending`
- `GET /api/v1/catalog/filters`

The API cases cover:

- smoke validation
- happy-path catalog queries
- validation and negative inputs
- boundary values
- detail retrieval
- not-found behavior
- lookup endpoints

All API cases were derived from real routes only. No undocumented endpoint was added.

[INSERT TABLE D-3: sample API test case rows from `test-cases/api/OSMS-API-Test-Cases.xlsx`]

3.4 Test Data

The project uses seeded credentials and curated datasets to avoid generic placeholder data. Real reusable test data includes:

- valid seeded accounts for `admin`, `sales`, and `warehouse`
- invalid credential combinations
- category, brand, and product identifiers observed from the running system

- API query variations for positive and negative requests
- product import workbooks with mixed-validation rows
- invalid upload samples for attachment and file-format testing

Sensitive live credentials were not committed. Only demo-safe seeded data and controlled testing datasets were included in the repository.

[INSERT TABLE D-4: test-data summary from `test-data/accounts/OSMS-Test-Accounts.md` and `test-data/ui/OSMS-UI-Test-Data.xlsx`]

3.5 Automation Design And Implementation

To improve rubric score and bonus potential, a limited automation scope was implemented with maintainability in mind:

- UI automation stack: `.NET 8 + xUnit + Selenium WebDriver`
- API automation stack: `Postman + Newman`
- design pattern: `Page Object Model`
- reusable support layer for:
  - configuration
  - CSV test-data loading
  - waits
  - screenshots
  - webdriver management

The selected UI automation flows were:

- admin login smoke
- sales access-denied navigation
- privileged-user draft purchase creation
- privileged-user invoice creation
- product import preview

The selected API automation groups were:

- health smoke
- catalog happy path
- catalog validation and negative cases
- product detail
- trending and filters

[INSERT TABLE D-5: automation scope summary from `docs/phase-7-automation-design.md`]

3.6 Execution Summary

As of `2026-04-06`, the real execution set is materially stronger than the initial partial run, but it still must be interpreted conservatively:

- total test cases: `63`
- executed: `25`
- pass: `24`
- fail: `1`
- blocked: `0`

- not run: `38`
- execution progress: `39.68%`

The strongest confirmed pass results now include:

- `TC-UI-AUTH-001` - admin login smoke
- `TC-UI-AUTH-003` - denied navigation to purchases is enforced by redirect away from the protected route
- `TC-UI-IMP-002` - product import preview counts are displayed correctly
- `TC-UI-PUR-001` and `TC-UI-PUR-007` - draft purchase creation and details verification
- all `19` API cases - full Newman collection pass

The current confirmed fail result is:

- `TC-UI-INV-001`
  - linked defect: `BUG-20260406-001`
  - root cause confirmed by UI evidence and ASP.NET Core server log excerpt

[INSERT TABLE E-1: final result summary from `results/OSMS-Final-Test-Results.xlsx`]
[INSERT TABLE E-2: metrics summary from `metrics/OSMS-Test-Metrics.xlsx`, sheet `Summary`]

3.7 Execution Evidence

The current evidence set proves that both UI and API automation are functioning beyond a smoke-only baseline:

- UI evidence
  - login success screenshot exists for `TC-UI-AUTH-001`
  - permission-denial behavior screenshot exists for `TC-UI-AUTH-003`
  - import preview-count screenshot exists for `TC-UI-IMP-002`
  - purchase details screenshot exists for `TC-UI-PUR-001` and `TC-UI-PUR-007`
  - invoice failure screenshot exists for `TC-UI-INV-001`
- API evidence
  - full Newman collection output exists for all `19` API requests
- server-log evidence
  - extracted defect log exists for `BUG-20260406-001`

Figure E-1. Admin login success evidence.

Figure E-2. Newman full-run summary snippet.

Figure E-3. Draft purchase creation success evidence.

```
![Figure E-3 - Draft purchase
created](../evidence/ui/automation/20260406_054004_TC-UI-PUR-001-draft-
created.png)

Figure E-4. Invoice creation failure evidence.

![Figure E-4 - Invoice creation
failure](../evidence/ui/automation/20260406_053902_TC-UI-INV-001-
failure.png)

Figure E-5. Permission-denial redirect evidence for sales user.

![Figure E-5 - Permission
denial](../evidence/ui/automation/20260406_054245_TC-UI-AUTH-003-access-
denied.png)

Figure E-6. Product import preview-count evidence.

![Figure E-6 - Product import
preview](../evidence/ui/automation/20260406_054115_TC-UI-IMP-002-
preview.png)

# IV. Defect Report And Metrics

## 4.1 Defect Log

At the current reporting date, the repository contains `1` confirmed
product defect and `0` open observations pending manual confirmation. The
two older automation observations were closed after focused reruns proved
that they were automation or expectation issues rather than product
defects.

Current confirmed defect record:

- `BUG-20260406-001`
  - related to `TC-UI-INV-001`
  - module: `Invoices`
  - severity / priority: `High / High`
  - current state: `Open - Confirmed`
  - GitHub Issue: `#1`
  - GitHub URL: `https://github.com/Alucard30Dec/Online-Sales-Management-
System/issues/1`
  - evidence:
    - UI failure screenshot
    - focused rerun TRX
    - extracted server log excerpt showing `InvalidOperationException` in
`InvoicesController.Create`
    - GitHub Issue screenshot with labels

[INSERT TABLE B-1: `defects/exports/OSMS-Defect-Register.csv`]

Figure B-1. GitHub Issue evidence for the confirmed invoice defect.
```

![Figure B-1 - GitHub issue screenshot](../evidence/defects/BUG-20260406-001-github-issue.png)

4.2 Defect Management Workflow

The defect-management process was prepared using a GitHub-compatible workflow with explicit labels for:

- severity
- priority
- status
- module
- interface
- defect type

The intended workflow is:

1. execute or re-execute a related test case
2. classify the result as `Confirmed Defect`, `Observation`, or `Automation Script Failure`
3. open a GitHub Issue only after a defect is reproducible
4. attach screenshots, runner output, and expected-versus-actual details
5. retest before closure

[INSERT TABLE B-2: label taxonomy summary from `defects/github-issues/github-label-taxonomy.md`]

4.3 Test Summary Metrics

The current metrics show that the submission is structurally strong in planning and traceability, but still early in real execution depth.

Key metrics:

- executed test cases: `25 / 63`
- pass rate on executed cases: `96%`
- fail rate on executed cases: `4%`
- blocked rate on executed cases: `0%`
- documented scenarios: `42`
- mapped scenarios: `42`
- scenario execution coverage: `26.19%`

Interface-wise view:

- `Admin UI`
  - total: `41`
  - executed: `6`
  - pass: `5`
  - fail: `1`
- `Public UI`
  - total: `3`
  - executed: `0`
  - pass: `0`
  - fail: `0`

- `API`
  - total: `19`
  - executed: `19`
  - pass: `19`

Module-wise view shows that the public API surface is fully executed, `Purchases` and `Product Import` now have positive evidence, and `Invoices` contains the current confirmed defect. `Stock`, `Reports`, `Products`, and `Public Catalog` remain largely unexecuted.

[INSERT TABLE M-1: `metrics/OSMS-Interface-Results.csv`]
[INSERT TABLE M-2: `metrics/OSMS-Module-Wise-Results.csv`]
[INSERT TABLE M-3: `metrics/OSMS-Scenario-Coverage.csv`]

Figure M-1. Metrics summary exported from the workbook.

V. Conclusion And Future Work

5.1 Achievements And Compliance

This submission achieved the following:

- completed a source-based project audit rather than a generic theory-based plan
- defined a structured test plan, scenario set, UI/API test cases, and reusable test data
- implemented a real automation framework for UI and API testing
- captured real execution evidence for both UI and API
- prepared a professional defect-management package and metrics pack
- maintained clear traceability between scenarios, test cases, results, evidence, and observations

5.2 Challenges And Limitations

The main limitation of the current package is no longer only execution depth. While `25` of `63` test cases now have real evidence, the system still contains one confirmed high-severity defect in invoice creation, and several business UI areas remain unexecuted. Because of this, the report cannot responsibly claim that the full system is stable.

The report still lacks some high-value execution depth:

- additional business-flow evidence for `Stock`, `Reports`, `Products`, and `Public Catalog`

5.3 Future Enhancements

The highest-priority next actions are:

1. fix and retest invoice creation, then add post-fix evidence to GitHub Issue `#1`

2. execute the remaining high-value UI areas: `Stock`, `Reports`, `Products`, and `Public Catalog`
3. optionally add cross-browser evidence if Edge is available
4. keep the final PDF, PPTX, issue screenshot, and automation video aligned with any future retest update

5.4 Final Conclusion

Based on the current execution evidence, the `Online Sales Management System` has a verified baseline for admin login, permission enforcement, product import preview, draft purchase creation, and the full public API surface. However, it does not yet have enough executed UI coverage to support a broad claim of operational stability, and invoice creation currently contains one confirmed high-severity defect. The most defensible conclusion is that the project is partially verified, strong in design and traceability, and close to a stronger final submission once the remaining UI executions, GitHub issue evidence, and automation video are completed.

References

1. `Report Test subject/UEF - Final.pdf`
2. `Report Test subject/docs/phase-0-project-audit.md`
3. `Report Test subject/docs/test-plan.md`
4. `Report Test subject/docs/test-scenarios.md`
5. `Report Test subject/docs/phase-7-automation-design.md`
6. `Report Test subject/docs/phase-9-execution-and-evidence.md`
7. `Report Test subject/docs/phase-10-defect-log-and-bug-management.md`
8. `Report Test subject/docs/phase-11-final-results-and-metrics.md`
9. `Report Test subject/docs/phase-12-analysis-and-insights.md`
10. Source repository: `https://github.com/Alucard30Dec/Online-Sales-Management-System`

Appendix

Appendix A. GitHub Submission Link

- `https://github.com/Alucard30Dec/Online-Sales-Management-System/tree/main/Report%20Test%20subject`

Appendix B. Key Attached Artifacts

- UI test cases: `test-cases/ui/OSMS-UI-Test-Cases.xlsx`
- API test cases: `test-cases/api/OSMS-API-Test-Cases.xlsx`
- UI test data: `test-data/ui/OSMS-UI-Test-Data.xlsx`
- API test data: `test-data/api/OSMS-API-Test-Data.json`
- Automation scripts: `automation/`
- Final results: `results/OSMS-Final-Test-Results.xlsx`
- Metrics: `metrics/OSMS-Test-Metrics.xlsx`
- Defect log: `defects/exports/OSMS-Defect-Log.xlsx`
- UI evidence: `evidence/ui/automation/`
- API evidence: `results/automation-api/`
- Video: `video/OSMS-Automation-Demo.mp4`

Appendix C. Mandatory Pending Items Before Strong Final Submission

- `PENDING REAL EXECUTION`: additional execution evidence for `Stock`, `Reports`, `Products`, and `Public Catalog`